



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

[Git] Realizando alterações

Profa. Lana Mesquita



Conteúdo da aula

- Realizando alterações
- git add
- git config
- git commit
- git log
- git diff
- Ignorando arquivos

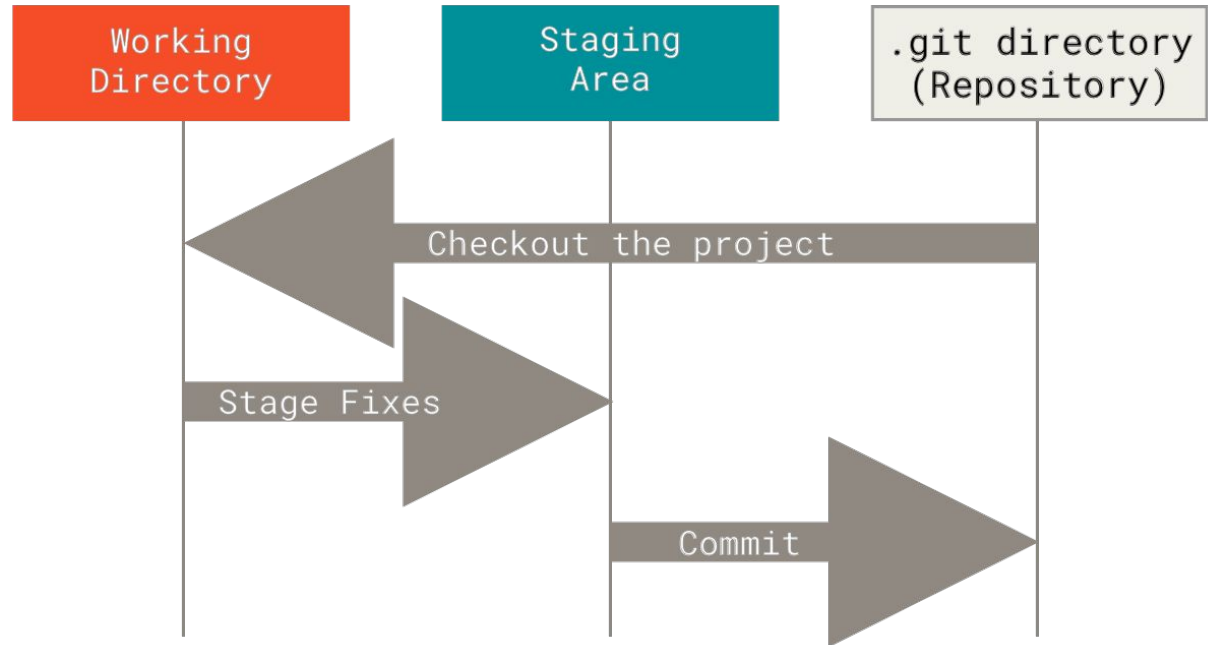
Realizando alterações

Os 3 estados

O Git tem três estados principais que seus arquivos podem estar: committed, modificado (modified) e preparado (staged).

- **Committed** significa que os dados estão armazenados de forma segura em seu banco de dados local.
- **Modificado (modified)** significa que você alterou o arquivo, mas ainda não fez o commit no seu banco de dados.
- **Preparado (staged)** significa que você marcou a versão atual de um arquivo modificado para fazer parte de seu próximo commit

Os 3 estados

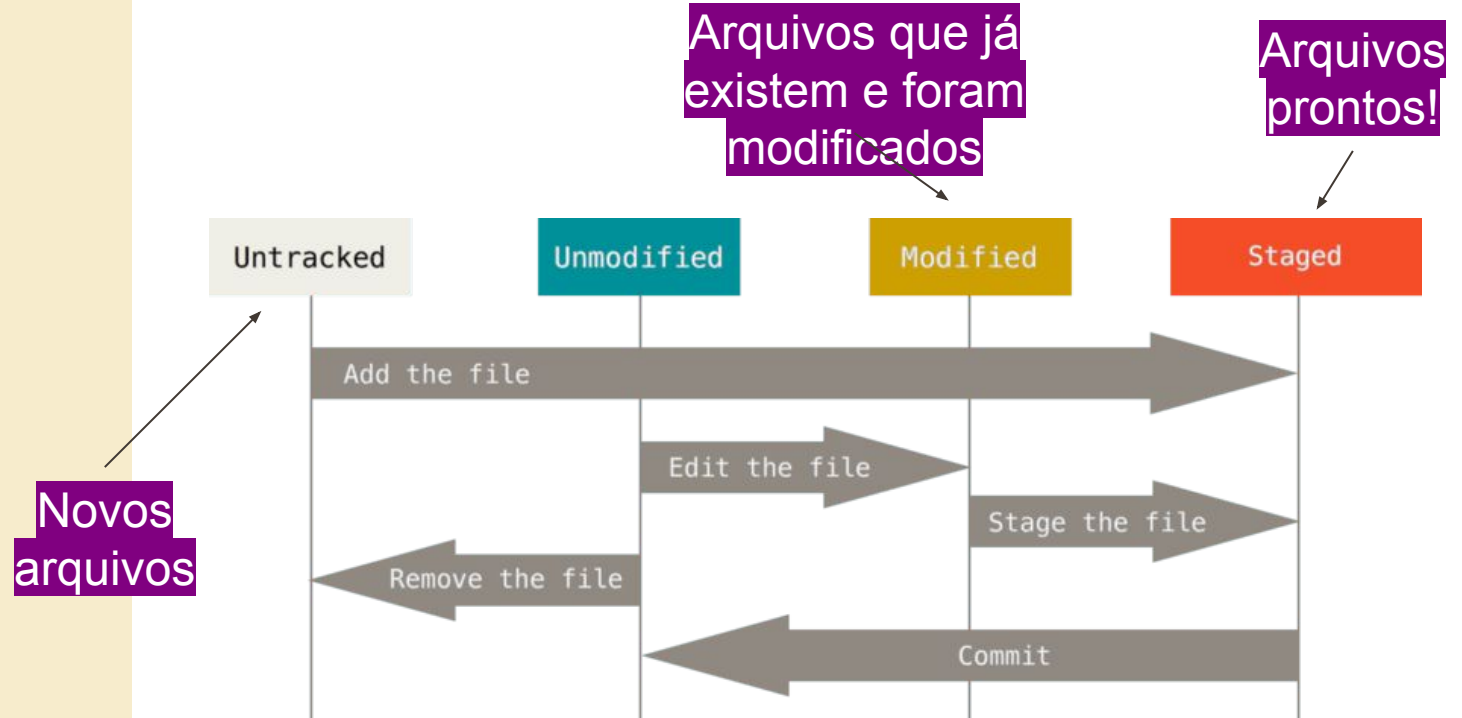


Fluxo de trabalho no Git

O fluxo de trabalho básico Git é algo assim:

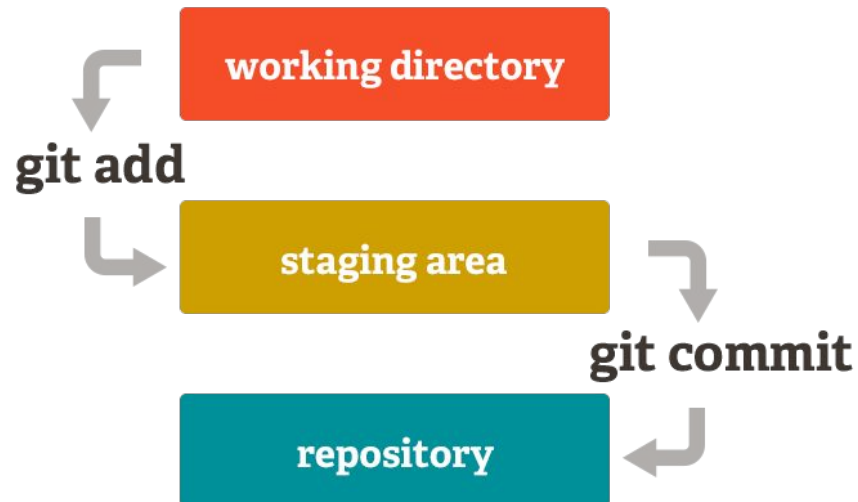
1. Você **modifica** arquivos no seu diretório de trabalho.
2. Você **prepara** os arquivos, adicionando imagens deles à sua área de preparo.
3. Você faz **commit**, o que leva os arquivos como eles estão na área de preparo e armazena essa imagens de forma permanente para o diretório do Git.

Áreas de trabalho no Git



Definições

- **Working tree:** Local onde os arquivos realmente estão sendo armazenados e editados (arquivos *untracked*)
- **Index (*staging area*):** Local onde o Git armazena o que será commitado, ou seja, o local entre a working tree e o repositório Git em si.

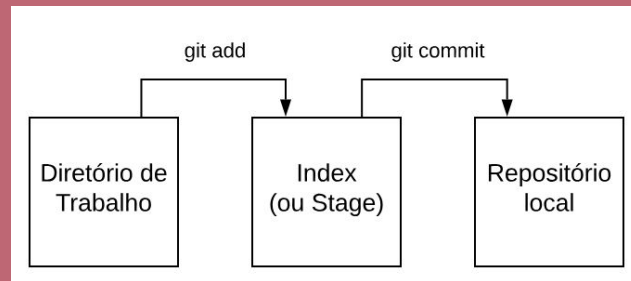


Boas práticas

- Realizem **commits pequenos periodicamente**, sempre que tiverem efetuado uma mudança importante no código.
- **Não incluir modificações relativas a mais de uma tarefa** de manutenção. Por exemplo, não é recomendável corrigir dois bugs em um mesmo commit, para facilitar análise do código.
- **Nunca devemos ter commits de códigos que não funcionam**, mas também não é interessante deixar para commitar apenas no final de uma feature.

**Quais os
dados de um
commit?**

<https://github.com/excalidraw/excalidraw/commit/ae89608985dfe5975b5fa0103d5aec64cb4fb83f>

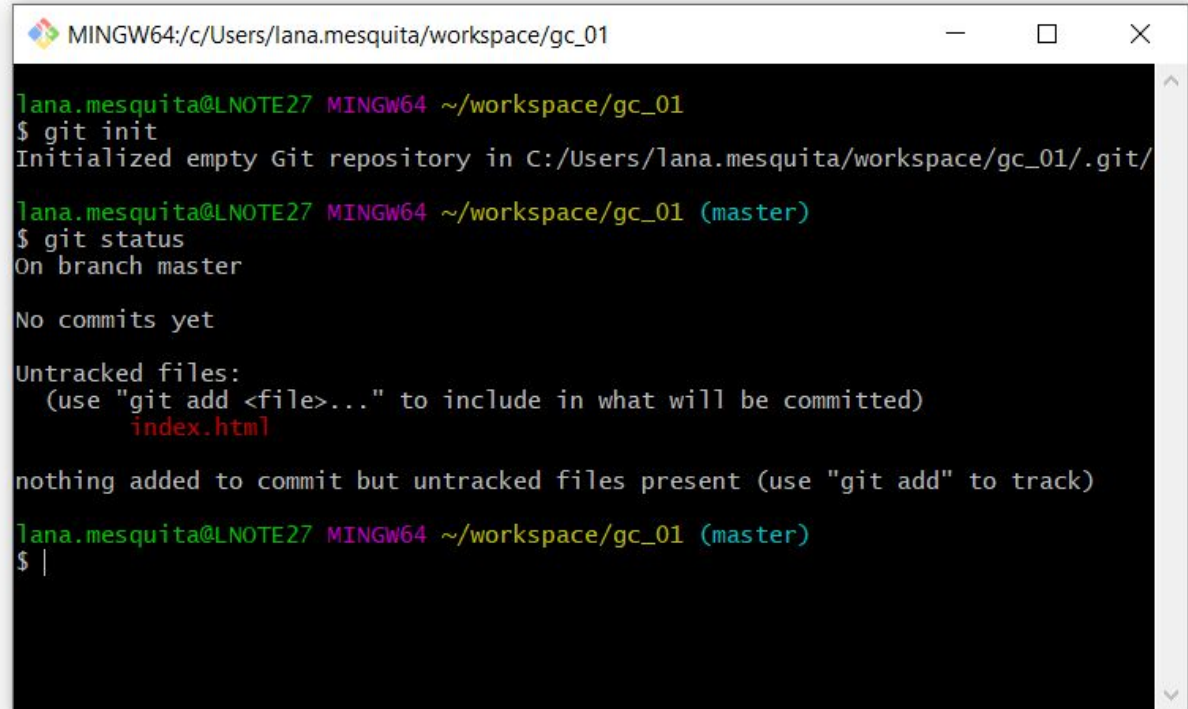


git add

<https://github.com/git-guides/git-add>

Aula passada

Git status



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01
$ git init
Initialized empty Git repository in C:/Users/lana.mesquita/workspace/gc_01/.git/

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git status
On branch master

No commits yet

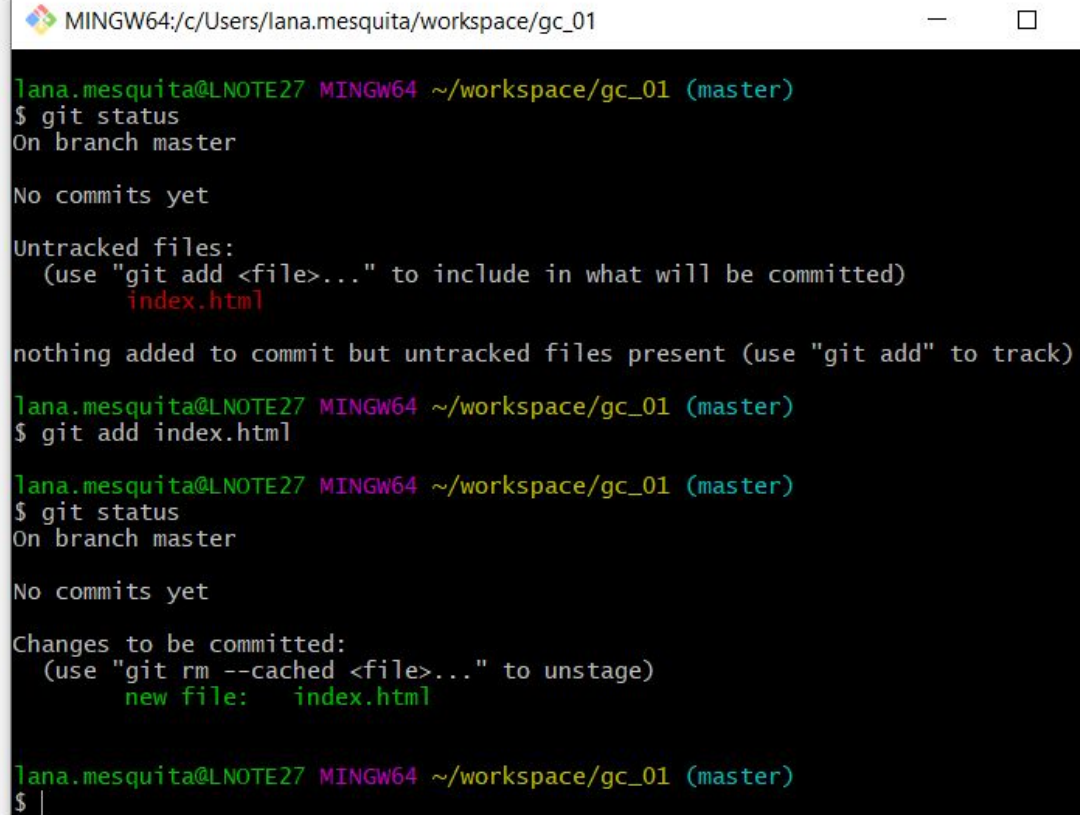
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ |
```

Gerenciando alterações

Git add
+ git status



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git add index.html

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git status
On branch master

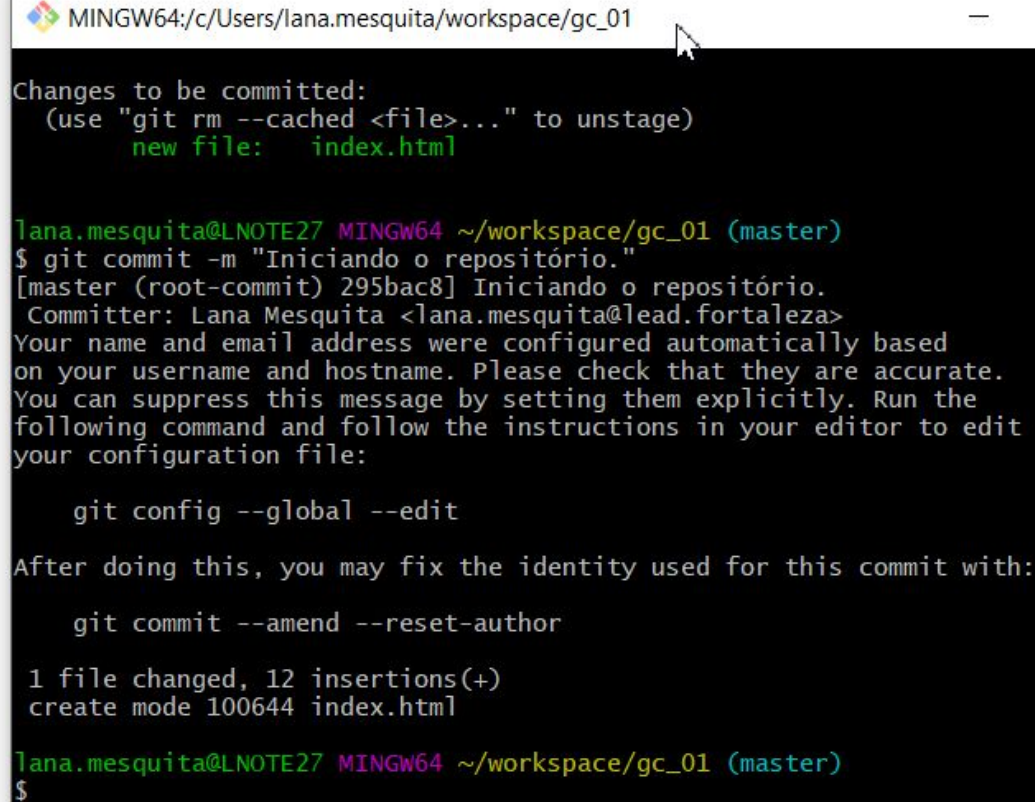
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ |
```

Gerenciando alterações

~~Git commit~~
git config...



```
MINGW64:/c:/Users/lana.mesquita/workspace/gc_01

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   index.html

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git commit -m "Iniciando o repositório."
[master (root-commit) 295bac8] Iniciando o repositório.
  Committer: Lana Mesquita <lana.mesquita@lead.fortaleza>
  Your name and email address were configured automatically based
  on your username and hostname. Please check that they are accurate.
  You can suppress this message by setting them explicitly. Run the
  following command and follow the instructions in your editor to edit
  your configuration file:

      git config --global --edit

  After doing this, you may fix the identity used for this commit with:

      git commit --amend --reset-author

  1 file changed, 12 insertions(+)
  create mode 100644 index.html

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```

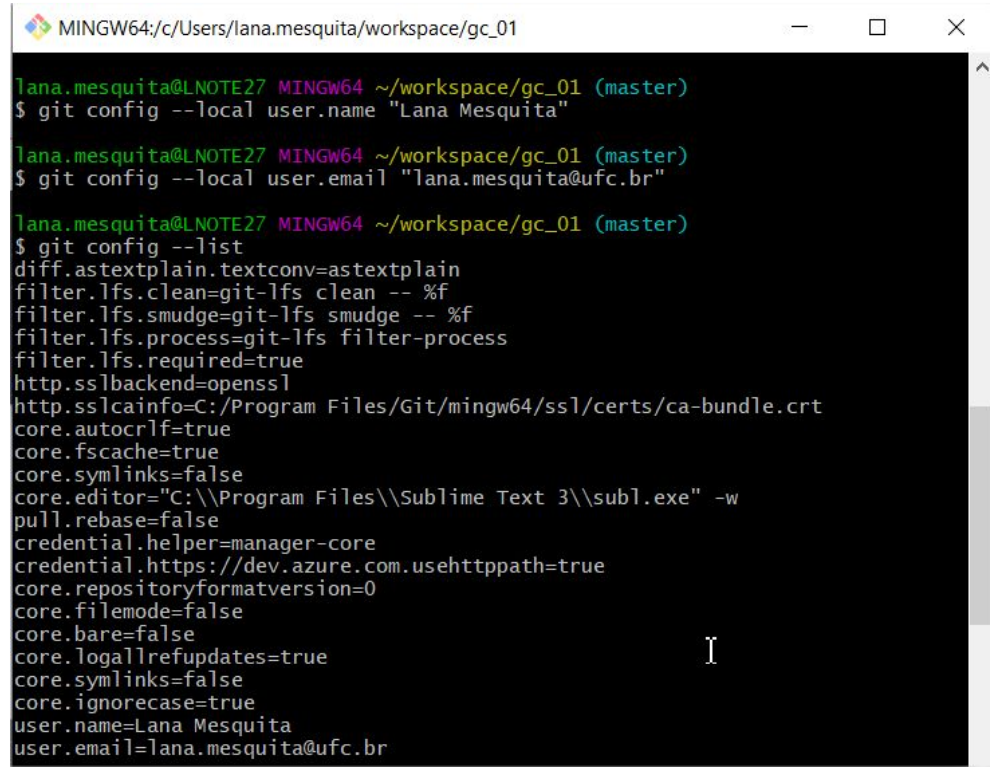
git config

<https://git-scm.com/book/pt-br/v2/Come%C3%A7ando-Configura%C3%A7%C3%A3o-Inicial-do-Git>



Configuração git

Se tiver usado a opção **--global** em vez de **--local** o Git usará esta informação para qualquer coisa que você fizer naquele sistema.



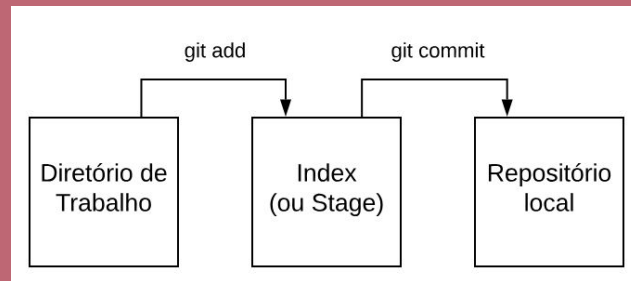
```
MINGW64/c:/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git config --local user.name "Lana Mesquita"

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git config --local user.email "lana.mesquita@ufc.br"

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
core.editor="C:\\Program Files\\Sublime Text 3\\subl.exe" -w
pull.rebase=false
credential.helper=manager-core
credential.https://dev.azure.com.usehttppath=true
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Lana Mesquita
user.email=lana.mesquita@ufc.br
```

```
git config --local user.name "Seu nome aqui"
git config --local user.email "seu@email.aqui"
```

git commit

<https://github.com/git-guides/git-commit>

Commit

- Coleção de alterações realizadas, espécie de "checkpoint" do projeto.
- Sempre que necessário você pode **retroceder** a algum commit.
- **É possível fazer commits à vontade no seu clone**, e só publicar ou enviar essas informações para um servidor (se houver) posteriormente.
- Você pode fazer **commits menores** e mais frequentes, mesmo quando ainda está testando o seu código, pois você não vai estar commitando código não testado num repositório central.

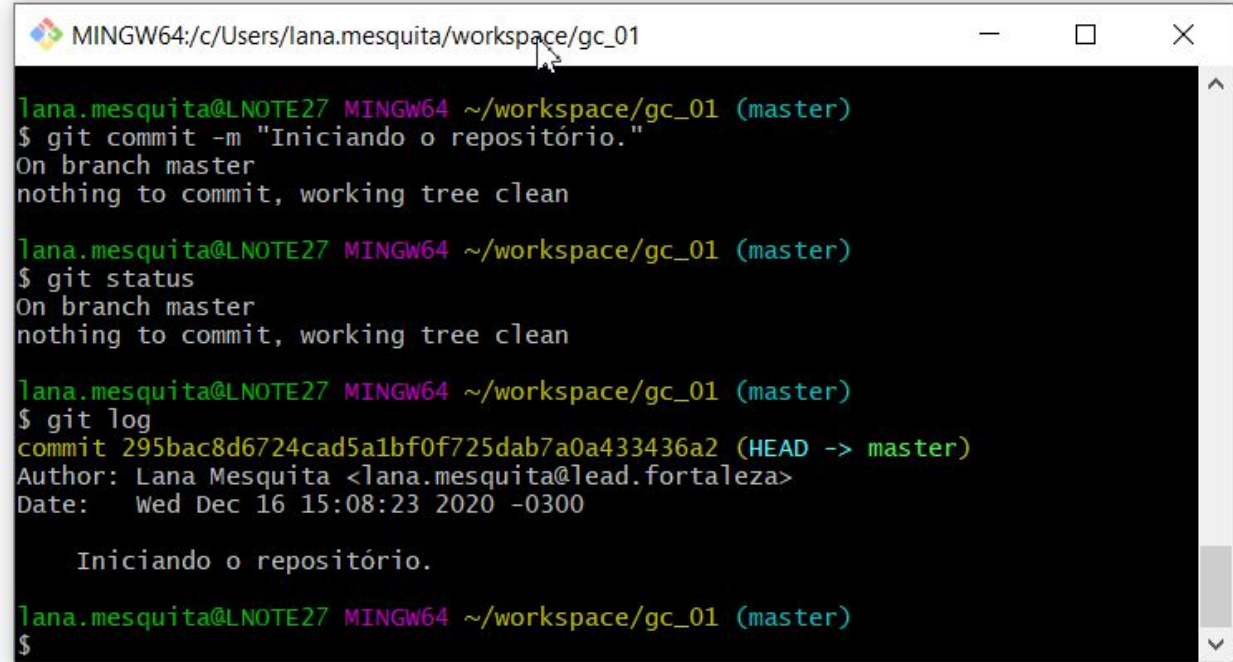
Gerenciando alterações

Git commit

git status

git log

Dica Git Bash: utilize as setas para cima e baixo para repetir comandos anteriores.



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git commit -m "Iniciando o repositório."
On branch master
nothing to commit, working tree clean

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git status
On branch master
nothing to commit, working tree clean

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log
commit 295bac8d6724cad5a1bf0f725dab7a0a433436a2 (HEAD -> master)
Author: Lana Mesquita <lana.mesquita@lead.fortaleza>
Date:   Wed Dec 16 15:08:23 2020 -0300

    Iniciando o repositório.

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```

Como boa prática, a mensagem descritiva de commit deve ser clara e não deve ser muito grandes.

**Você fez um commit e
parou num lugar esquisito?**

Editor padrão no git commit sem o -m

O git commit sem o opcional **-m** para a mensagem irá abrir seu editor de texto para você criar a mensagem de commit.

Se você não configurou nada, há uma boa chance de ser o VI ou o Vim. (Para sair, pressione Esc, depois :wq, e depois Enter.)



VIM

O Vim tem alguns modos:

- Normal mode: para se movimentar pelo texto
- Insert mode: para escrever no texto
- Visual mode: para selecionar parte do texto
- Command mode: para dar algum comando ao Vim

Alguns comandos úteis:

- i: entrar no modo insert
- esc: sair do modo atual e ir para o modo de comando
- :q: sair do arquivo atual
- :q!: saída forçada do arquivo atual, sem salvar
- :w: salvar
- :wq: salvar e sair

Dica:

Aperte esc.
Digite :wq
Pronto!

Mais comandos: <https://dev.to/nfo94/comandos-basicos-do-vim-e-configuracoes-uteis-gkn>

git log

Lista informações sobre os últimos commits, como data, autor, hora e descrição do commit.

Gerenciando alterações

Git log



hash do commit

```
lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log
commit 295bac8d6724cad5a1bf0f725dab7a0a433436a2 (HEAD -> master)
Author: Lana Mesquita <lana.mesquita@lead.fortaleza>
Date:   Wed Dec 16 15:08:23 2020 -0300

    Iniciando o repositório.

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```

HEAD: Estado atual do nosso código, ou seja, onde o Git nos colocou

Hash do commit

Os 7 primeiros caracteres do hash geralmente são suficientes para operações, já que a colisão de hashes é extremamente rara.

Cada commit no Git recebe um **hash único**, que é uma string de **40 caracteres hexadecimais** gerado usando o algoritmo de hash SHA-1 e mudando aos poucos para SHA-256, para evitar ataques de colisão.

Serve como a **impressão digital** do commit, garantindo que cada mudança na história seja identificável.

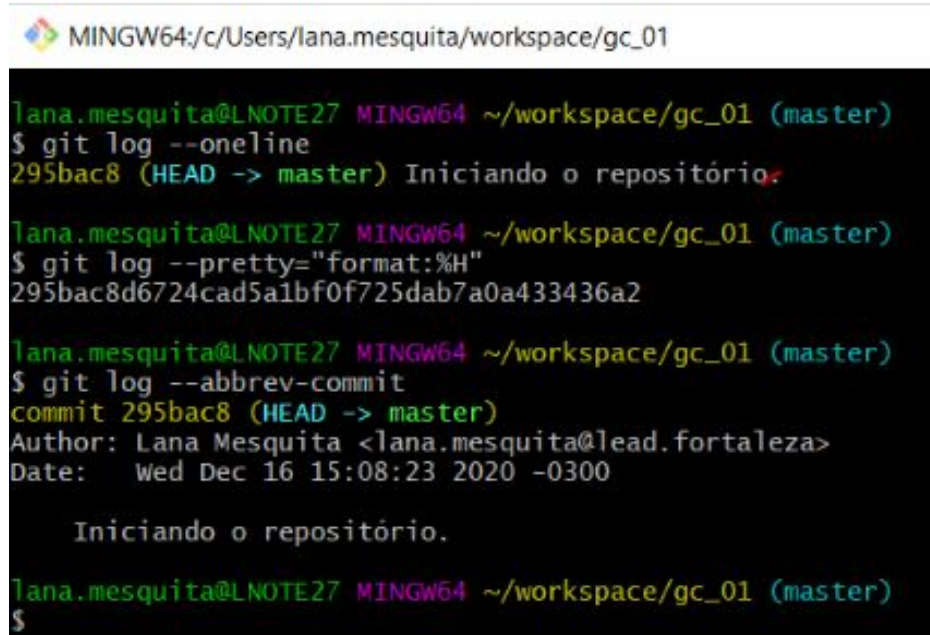
Garante **Integridade dos Dados**: o hash é gerado a partir do conteúdo do commit, incluindo a mensagem, autor, carimbo de data/hora e o hash do commit pai.

Rastreabilidade e Navegação: Permite rastrear com precisão o histórico de versões do projeto, fazer, por exemplo, *revert*, *reset* ou *merge*.

git log

git log --oneline

git log --pretty="format:%H"



```
MINGW64:/c:/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log --oneline
295bac8 (HEAD -> master) Iniciando o repositório.

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log --pretty="format:%H"
295bac8d6724cad5a1bf0f725dab7a0a433436a2

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log --abbrev-commit
commit 295bac8 (HEAD -> master)
Author: Lana Mesquita <lana.mesquita@lead.fortaleza>
Date:   Wed Dec 16 15:08:23 2020 -0300

    Iniciando o repositório.

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```

git log -p

git log --oneline

git log --pretty="format:%H"

MINGW64:/c/Users/lana.mesquita/workspace/gc_01

```
lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log --oneline
295bac8 (HEAD -> master) Iniciando o repositório.

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log --pretty="format:%H"
295bac8d6724cad5a1bf0f725dab7a0a433436a2

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git log --abbrev-commit
commit 295bac8 (HEAD -> master)
Author: Lana Mesquita <lana.mesquita@lead.fortaleza>
Date:   Wed Dec 16 15:08:23 2020 -0300

    Iniciando o repositório.

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```

Mais opções:

<https://git-scm.com/book/pt-br/v2/Fundamentos-de-Git-Vendo-o-hist%C3%B3rico-de-Commits>

`git diff`

Comparar modificações.

git diff

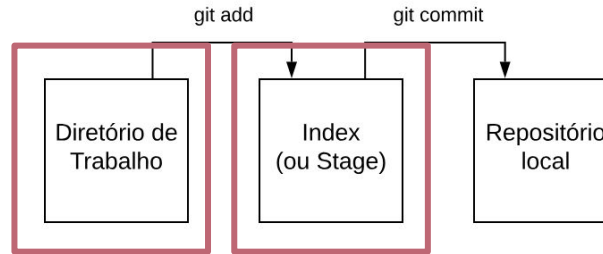
O git diff responde às perguntas:

- O que você alterou mas ainda não mandou para o stage (estado preparado)?
- E o que está no stage, pronto para o commit?

O git status lista os nomes dos arquivos, o git diff exibe exatamente as linhas que foram adicionadas e removidas — o *patch*, como costumava se chamar.

git diff

alterações não
enviadas para
stage



```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

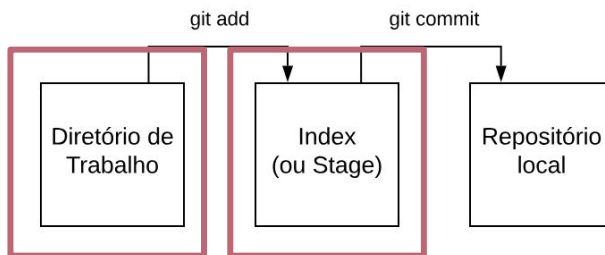
    modified:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

git diff

alterações não
enviadas para
stage



Use só

git diff

```
$ git diff
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 8ebb991..643e24f 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -65,7 +65,8 @@ branch directly, things can get messy.
 Please include a nice description of your changes when you submit your PR;
 if we have to read the whole diff to figure out why you're contributing
 in the first place, you're less likely to get feedback and have your change
-merged in.
+merged in. Also, split your changes into comprehensive chunks if your patch is
+longer than a dozen lines.
```

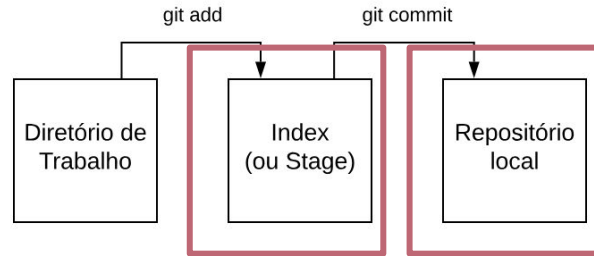
If you are starting to work on a particular area, feel free to submit a PR that highlights your work in progress (and note in the PR title that it's

git diff

alterações do
stage ainda não
commitadas

Use

```
git diff --staged  
git diff --cached
```



```
$ git diff --staged  
diff --git a/README b/README  
new file mode 100644  
index 0000000..03902a1  
--- /dev/null  
+++ b/README  
@@ -0,0 +1 @@  
+My Project
```



```

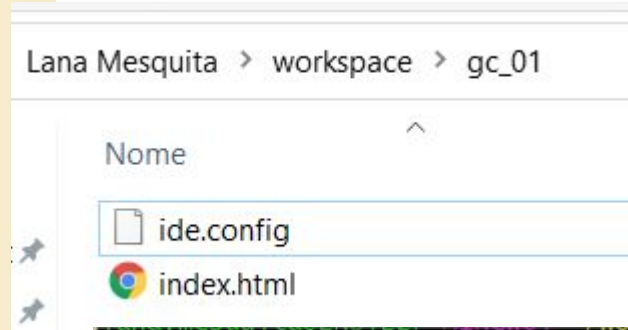
packages/element/src/resizeElements.ts
@@ -35,6 +35,7 @@ import {
  35  35     getContainerElement,
  36  36     handleBindTextResize,
  37  37     getBoundTextMaxWidth,
  38  38     computeBoundTextPosition,
  39  39 } from "../textElement";
  40  40 import {
  41  41     getMinTextElementWidth,
@@ -225,7 +226,16 @@ const rotateSingleElement = (
  225  226     scene.getElement<ExcalidrawTextElementWithContainer>(boundTextElementId);
  226  227
  227  228     if (textElement && !isArrowElement(element)) {
  228  228     -     scene.mutateElement(textElement, { angle });
  229  229     +     const { x, y } = computeBoundTextPosition(
  230  230     +         element,
  231  231     +         textElement,
  232  232     +         scene.getNonDeletedElementsMap(),
  233  233     +     );
  234  234     +     scene.mutateElement(textElement, {

```

Ignorando arquivos

Ignorando arquivos

- Alguns arquivos não devem ser monitorados
 - Ex. um arquivo de configurações da IDE ou do build do sistema



```
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
       ide.config

nothing added to commit but untracked files present (use "git add" to track)
```

Regras do .gitignore

As regras para os padrões que podem ser usados no arquivo .gitignore são as seguintes:

- Linhas em branco ou começando com # são ignoradas.
- Os padrões que normalmente são usados para nomes de arquivos funcionam.
- Você pode iniciar padrões com uma barra (/) para evitar recursividade.
- Você pode terminar padrões com uma barra (/) para especificar um diretório.
- Você pode negar um padrão ao fazê-lo iniciar com um ponto de exclamação (!).

Regras do .gitignore

Padrões de nome de arquivo são como expressões regulares simplificadas usadas em ambiente shell:

- Um asterisco ***** casa com zero ou mais caracteres;
- **[abc]** casa com qualquer caractere dentro dos colchetes (neste caso, a, b ou c);
- um ponto de interrogação **?** casa com um único caracter qualquer;
- e caracteres entre colchetes separados por hífen **[0-9]** casam com qualquer caracter entre eles (neste caso, de 0 a 9).

Veja:

<https://github.com/git-hub/gitignore>

Exemplo de gitignore

```
# ignorar arquivos com extensão .a
*.a

# mas rastrear o arquivo lib.a, mesmo que você esteja ignorando os arquivos .a acima
!lib.a

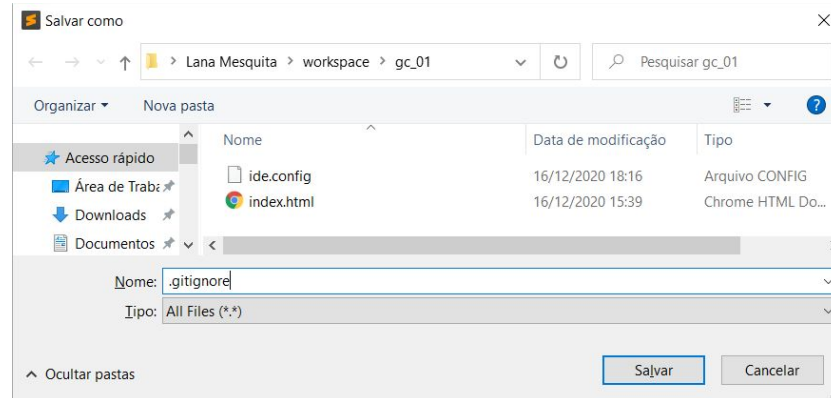
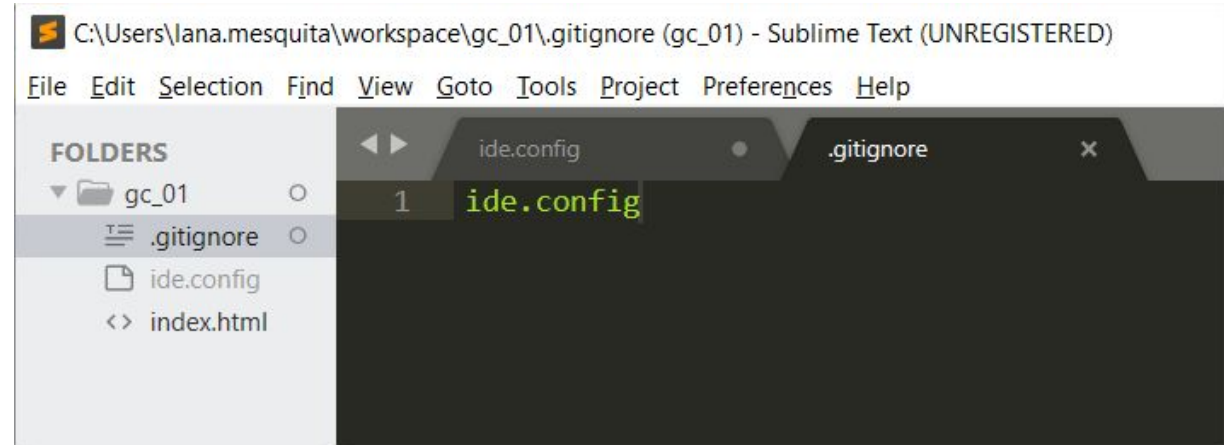
# ignorar o arquivo TODO apenas no diretório atual, mas não em subdir/TODO
/TODO

# ignorar todos os arquivos no diretório build/
build/

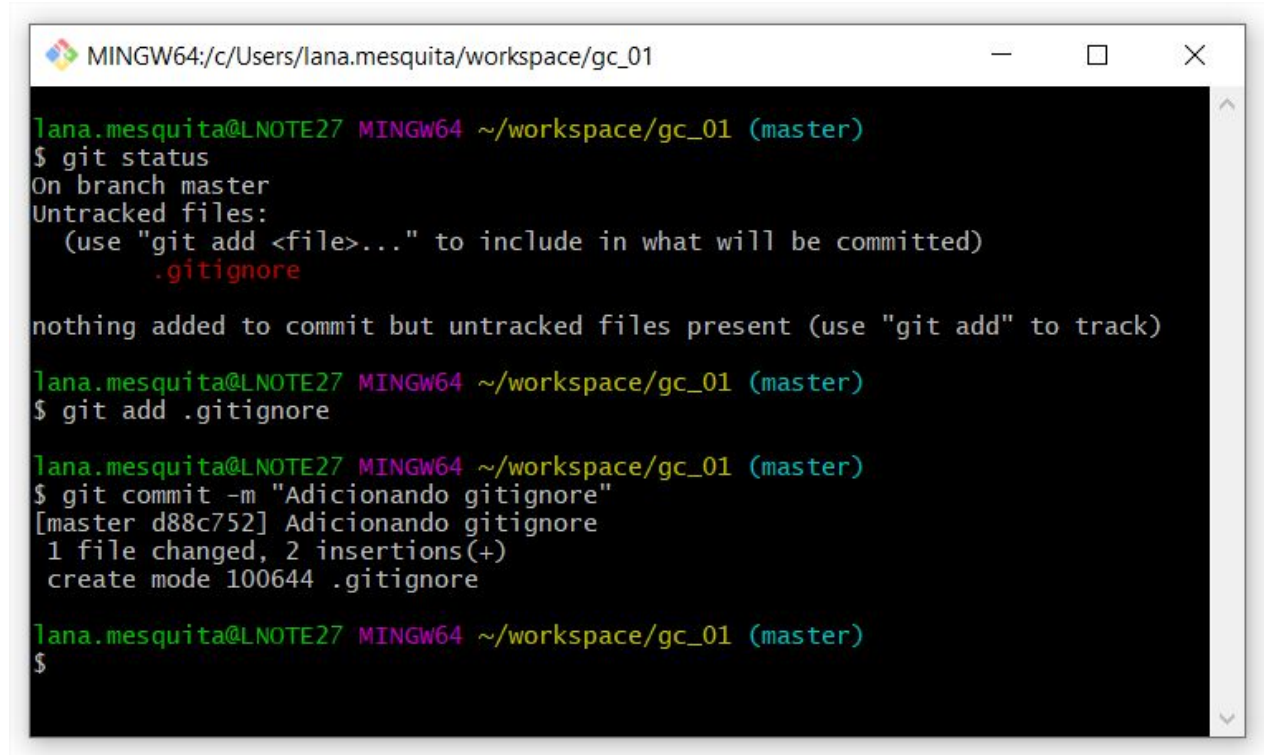
# ignorar doc/notes.txt, mas não doc/server/arch.txt
doc/*.txt

# ignorar todos os arquivos .pdf no diretório doc/
doc/**/*.pdf
```

.gitignore



.gitignore



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git add .gitignore

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git commit -m "Adicionando gitignore"
[master d88c752] Adicionando gitignore
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```


Dúvidas?
Mandem por Discord!



Até a próxima aula!

Lana Mesquita

